

(T1) Assessment Brief CMP4266

Module Title: Computer programming			
Module Code: CMP4266			
Assessment Type (delete as appropriate) Coursework	Level 4	Weighting 100%	Word Count
Submission Date Quiz Week 4 Weekly Submissions Coursework 12 January 2026	Submission Time 3 pm	Module Leader Dr. Ogerta Elezaj	Time Limit (for in person or oral assessments)

Assessment Brief

Assessment Information	
Assessment Task (with genre/type)	This assessment comprises of three components. 1. Weekly Submission of Lab Exercises (Total weight 30%) 2. Online Quiz on Week 4 (Total weight 20%) 3. Computer programme solution and testing (Total weight 50%). • This must include the following parts: • Design documentation submitted online (Weight 10%). • This must include the two flow chart diagrams. • Programme source code submitted online (Weight 30%). • Testing and evaluation inclusive of test cases submitted • online. (Weight 10%). This is a 100% coursework (portfolio) assignment
Assessment Title	Portfolio of Programming activities
Things to include:	See description of the task further below in this document.

Completion of this assessment will address the following learning outcomes:	
1	Apply problem solving skills to design solutions to programming problems.
2	Implement solutions in an imperative programming language by using common programming tools (such as editors and interpreters)
3	Use common programming tools and techniques (e.g. IDE, testing APIs and theories) to test, evaluate programs and document findings effectively in appropriate format.

Task: Python Coding Practice
Style: Summative Assessment

Rationale: This programming assignment is to apply the programming principles covered in tutorials and lectures to develop a python software that implements core hospital management solution that is used in by its patients and staff members. The software can be fully implemented using text-based interface, however, to achieve higher marks, GUI based software can be developed using the Python GUI Programming module named Tkinter. GUI design and Tkinter programming will be covered during week 10. The aim of the exercises is to enhance a student's experience of programming by applying programming principles to a larger problem of developing a complete application.

There are two reasons behind the selection of the hospital management system as the topic of this coursework. Firstly, the students are familiar with this system, hence, students will spend less time and effort to understand the functions of the software they will be developing for this coursework. Hence, most of their time will be devoted to the design, development and testing of the system by applying the programming knowledge and skills they learnt throughout this module. Secondly, GUI based programs make it easier to interact with the developed system and demonstrate a direct relationship between the user interaction and the system functions and data. Also, students will learn about how a product works entirely from the user's (or customer's) perspective and not from just a developer perspective. Hence, they will need to develop a user-friendly GUI.

Description:

1. Weekly Lab Submission Assessment Brief (Total weight 30%)

Two lab Exercises commencing W1-W6 will be randomly picked and will be marked at 30% of the overall mark for this module. For instructions and guidance on these tasks, see the information under each week's lab on Moodle. Submission details Weekly labs are to be submitted at the end of the lab session. Note: You can always seek help and support from your tutors and demonstrators regarding your lab exercises.

2. Online Quiz in Week 4 (Total weight 20%)

Assessment 2 is an individual interactive quiz to be conducted in Week 4, consisting of multiple choice, true/false, and fill in multiple gaps. Marking Criteria: the quiz score is based on the number of questions the student can get correctly. The student's score will be displayed on the teacher's screen immediately after the submission button is clicked.

3. Design, development and testing of a computer programme solution (Total weight 50%)

Part A – Design documentation (weight 10%)

Overview of hospital management systems In this coursework, students will create a 'prototype' Hospital Management system. This system aims to maintain the medical records of the patient, maintain the contact details of the patient and keep track of the appointment dates. There are three different types of users involved in a hospital management system: admin, doctor and patient. Each of the above plays an important role in the hospital management system performing different tasks. Patients using the system can book online appointments. Their appointments are approved by the admin that also assign a doctor to them. Use the above activities to draw two flowcharts using the special shapes to represent different types of actions or steps in a process. Link these activities with lines and arrows to show the sequence of the steps, and the relationships among them. For this part of the assessment, you need to submit two flow chart diagrams in PDF format (total weight 10%). To complete this you might refer to Week 2 learning materials. (4%) One diagram should depict the admin log-in process for the hospital management system (6%) Second diagram should explain a critical/complicated process of the system e.g., admin approving an appointment requested by a patient and assigning a doctor and informing both the patient and the doctor.

Part B – System development (weight 30%)

Important: This part of the assessment should not be attempted by students until you are provided with the partial implementation code which you must use as a starting point of your system, and this will be provided to you around Week 8 of this semester. Developing your hospital management system from scratch is not acceptable for this assessment and may lead to losing substantial number of marks. It is critical to use the Python 3.6 or a higher version of Python and "Spyder" to ensure that your submission runs correctly when marked by your tutor. The objective of this assessment is to develop python software that implements a core hospital management solution that is used in hospitals by its patients and hospital admins. The partial implementation of a prototype system will be developed during the practical sessions in the PC labs. However, for students to achieve higher marks, they should enhance the system functionalities by using text files to store the patient's related information.

In the system we have one active role, that is Admin. Doctors and Patients are passive objects. In this system only Admin dashboard will be developed, and Patients and Doctors will act as data storage instances.

Assessment specifications with a detailed marking scheme:

- *To achieve a mark to maximum of 40% (12 marks) of the total marks for the System Development*

The partial code that is provided contains the following classes. You are given partial functions that allow the following tasks of the Admin class. For the Doctor and Patient, you are given partial functions that do some of the following tasks.

Admin

- ✓ Admin Login
- ✓ Register/view/update/delete doctor
- ✓ View patient
- ✓ Can assign doctor to a patient

Doctor

✓ View their patient details assigned to them by admin

Patient

✓ View the assigned doctor

- *To achieve a mark of 41% to maximum of 50%*

The application must implement all the above and the following:

Create the necessary classes and functions which allow admins to perform the following tasks:

- ✓ Discharge a patient account i.e. remove patient from the system when the treatment is done
- ✓ Admins can view the discharged patient list
- ✓ Update admin own information i.e. names and address
- ✓ Print all doctor details
- ✓ Print all patient details

- *To achieve a mark of 51% to maximum of 70% for the System Development*

The application must implement all the above and the following:

- ✓ Patients have names, symptoms, age, mobile, address etc.
- ✓ Patients of the same family are grouped together by their surname by Admin
- ✓ The hospital system should be able to store and load all patients' data from and into a file.

- *To achieve a mark of 71% to maximum 80% for the System Development*

The application must implement all the above and the following:

- ✓ Relocating patients from one doctor to another.
- ✓ Admins can request a management report. This should show the following information:
 - ❖ Total number of doctors in the system
 - ❖ Total number of patients per doctor
 - ❖ Total number of appointments per month per doctor
 - ❖ Total number of patients based on the illness type.

- *To achieve a mark of 80% and over for the System Development*

- ✓ View diagrams of the reports from i, ii, iii, and iv.
- ✓ Development of a suitable Graphical User Interface (GUI) to perform all the above functions.

Note: students need to submit their system alongside the necessary objects to test their software by the tutors when marking.

Part C – Testing and evaluation (weight 10%)

Software testing is the process of evaluating a software item to detect differences between given input and expected output. Students must choose inputs to exercise paths through the code and determine the appropriate outputs. Good testing means your system is free of logical and run time errors. For this part, you need to develop a test plan that includes test cases based upon the system design and implementation of your Hospital Management System. We expect around 10 unique and valid test cases for your system. Testing and evaluation mechanism and methods will be explained during Week 10 when we will cover “Debugging and Test Plan”. Testing and evaluation will only be done when the 1st version of your software has been implemented. All test tables should be submitted in PDF format.

Additional information:

The following links provide further information about the programming language used for teaching this module:

Essential Book:

Gaddis, T. (2015). Starting Out with Python. 3rd ed. Pearson Education

Recommended Tutorials:

- Portable Python (<http://www.portablepython.com>)
- Python Website (<http://www.python.org>)
- Python Official Documentation (<https://docs.python.org/3/>)
- Python Official Tutorial (<https://www.python.org/about/gettingstarted/>)

Recommended Readings:

- Downey, A. (2015). Think Python How to Think Like a Computer Scientist. 2nd ed. O'Reilly Media
- Shaw, Z. (2013). Learn Python the Hard Way. 3rd ed. Addison Wesley
- Spraul, V. A., (2012) Think Like a Programmer: An introduction to Creative Problem Solving, William Pollock

For advice on writing style, referencing and academic skills, please make use of the Centre for Academic Success: [Centre for Academic Success - student support | Birmingham City University \(bcu.ac.uk\)](http://bcu.ac.uk)

Transferable skills:

The assessment helps students to develop various transferable skills which are necessary to obtain a decent job in the future.

Problem Solving

Being a programmer involves the ability to identify real-world issues and coming up with a technological solution to address them. This requires having strong analytical skills to understand the issue and evaluate different solutions to find the one that best fits the problem.

Ability to express your thoughts effectively.

Using flowcharts provides an effective way of communicating the logic of a system to all people involved in the project regardless of their background and expertise. They can serve as an excellent bridge between software developers and end users.

Project evaluation skills

It is the process that critically examines a program. It involves collecting and analyzing information about program activities, characteristics, and outcomes in relation to customer specifications. Its purpose is to make judgements about a programme that you have developed, to improve its effectiveness and inform programming decisions.

Time management and planning

Design documentation serves as the basis for estimating, scheduling, and validating efforts.

Marking Criteria for Component 3 Computer programme solution and testing:

Table of Assessment Criteria and Associated Grading Criteria

Learning Outcomes	1 Design solutions to programming problems	2 Implement solutions in an imperative programming language by using common programming tools (such as editors and interpreters).	3 Use common programming tools and techniques (e.g. editors, testing APIs and theories) to test and evaluate programs
Assessment Criteria □			
Weighting:	10%	30%	10%
Grading Criteria 0 – 20% Fail	A basic attempt at understanding and expressing a possible internal design solution for the problem domain. Presented in text only, no attempt for diagrammatic presentation.	Significantly incomplete program. Does not compile and execute.	Little or no testing of the program.

20 – 39% Fail	A basic but relevant attempt at understanding and expressing a possible internal design solution for the problem domain. Presented in text only, no attempt for diagrammatic presentation.	Incomplete program. Does not compile and execute.	Weak testing that does not adequately test the programme features.
40 – 49%	Reasonable and relevant attempt at understanding and expressing a possible internal design solution for the problem domain. Diagrammatic presentation attempted, possibly incomplete and inaccurate.	Program compiles and demonstrate only few basic features of the design.	Basic testing of the program showing implemented.

50 – 59%	A good and relevant attempt at understanding and expressing a possible internal design solution for the problem domain. Diagrammatic presentation attempted. Possibly incomplete but accurate, or complete design attempted but inaccurate.	Program demonstrates most of the basic features of the design	Adequate testing of most of the basic the features of the program.	
60 – 64%	A good attempt at understanding and expressing a possible internal design solution for the problem domain.	Program demonstrates most of the basic features of the design and some other advanced features of the design.	Adequate testing of all features.	
65 – 69%	A very good attempt at understanding and expressing a possible internal design solution for the problem domain. Complete and accurate diagrammatic presentation.	Program demonstrates all of the basic features of the design and some other advanced features of the design.	Good testing of all features.	
70 – 79%	Excellent expression of a possible internal design solution for the problem domain. Complete and accurate diagrammatic presentation accompanied by clear documentation of the design.	Program demonstrates all the basic and advanced features of the design.	Very good testing of all features.	

80 – 89%	Excellent expression of a possible internal design solution for the problem domain. Complete and accurate diagrammatic presentation accompanied by clear documentation of the design.	Program demonstrate all the basic and some of the advanced features of the design. In addition, demonstrates some additional features relevant to the problem.	Very good testing of all features following the template.
90 – 100%	Excellent expression of a possible internal design solution for the problem domain. Complete and accurate diagrammatic presentation accompanied by clear documentation of the design. Discusses the wider issues in the problem domain that may impact the proposed design.	Program demonstrate all the basic and advanced features of the design. In addition, demonstrates all additional features relevant to the problem.	Excellent testing of all the features of the program.

Referencing Style

Please delete as appropriate

- BCU Harvard
- OSCOLA 4th Edition (Law)
- APA 7th Edition (Psychology)
- MHRA (Conservatoire Postgraduate Students)
- Chicago (English)
- IEEE (alternative format for Computing and Engineering Ph.D students)

More information on referencing is available here: <https://www.bcu.ac.uk/library/services-and-support/referencing>

Use of Artificial Intelligence

Whilst AI tools can be helpful in assisting learning, when it comes to assessment, the [Academic Misconduct Procedure](#) is clear that this should be a student's own original work and not the work of other people or AI tools.

The [Use of AI Tools – Student Guidelines](#) follows the same guidelines your lecturers use. If you are unsure of whether AI is appropriate within your work, please read the guidelines or ask your lecturer. For advice and guidance around academic writing, please visit the [Centre for Academic Success](#).

Academic Integrity Guidance

Academic Integrity means completing and submitting your own original work while properly acknowledging sources. Following academic conventions, such as referencing and citation, helps avoid plagiarism. The [Centre for Academic Success](#) offers resources on Moodle, workshops and 1-2-1 support to improve skills like paraphrasing and referencing.

Academic Misconduct

Academic misconduct is conduct that has or may have the effect of providing you with an unfair advantage by relying on dishonest means to gain advantage and which therefore compromises your academic integrity.

The [Academic Misconduct Procedure](#) sets out the process we will follow, and the penalties we may apply, in cases where we believe you may have compromised your academic integrity by committing academic misconduct.

Word Count

The maximum word count for this module assessment is shown on Page 1. A +10% margin of tolerance is applied, beyond which nothing further will be marked. Marks cannot be awarded for any learning outcomes addressed outside the word count.

The word count refers to everything in the main body of the text (including headings, tables, citations, quotes, lists etc.). Everything before (i.e. abstract, acknowledgements, contents, executive summaries etc.) and after (i.e. references, bibliographies, appendices etc) is **not** included in the word count limit.

Submission Information

Present any written aspects of the assessment using font size 11 and using 1.5 spacing to allow for comments and annotations to be added by the markers.

Complete the appropriate cover sheet for this assessment and append your work.

This assessment will be marked anonymously and should show your student number only.

Submit this coursework assessment task via Moodle.

Late Submission

BCU Assessment Brief Template (T1)

Where you are required to submit assessment by a certain deadline (for example essays, case studies or physical artifacts) but you fail to meet the deadline, your mark will be reduced in accordance with the [Late Submissions Policy](#). This Policy does not apply where the assessment is 'in-person' such as exams and in-class tests.

For support on submission, please review the [Submitting Your Assignment](#) e-learning resource.

----- *FOR STAFF ONLY* -----

Staff Use Only: Retained Work Record	
Details of Retained Work:	e.g. final written assignment for module XYZ
Location of Retained Work:	https://moodle.bcu.ac.uk/course/view.php?id=98582
Type of Work Retained (<i>tick all that apply</i>):	Written Photo Video Other (please provide details below) Other: